Министерство образования Республики Беларусь
Учреждение образования «Белорусский государственный университет информатики и радиоэлектроники»
Филиал «Минский радиотехнический колледж»

Учебный предмет
«Инструментальное программное обеспечение»

Отчёт
по выполнению лабораторной работы №14
«Разработка, отладка и испытание программ с применением наследования классов»

Выполнила: учащаяся гр. 4к9392
Кривнюк Д.А.
Проверила: Лазовская Н.В.

Минск,2025

Лабораторная работа №14

Тема работы: «Разработка, отладка и испытание программ с применением наследования классов»

1. Цель работы

Научить писать программы с применением наследования и множественного наследования.

2. Задание

К классу из лабораторной работы №13 добавить 3 класс которые будут является потомками уже имеющегося класса.

3. Оснащение работы

Задание по варианту, ЭВМ, среда разработки **Python 3.7, IDLE**.

4. Порядок выполнения работы

Код программы:

```
1. from abc import ABC, abstractmethod


class Car(ABC):
    def __init__(self, brand, color, mileage=0):
        self.brand = brand         self.color = color         self.mileage = mile-
age

    def info(self):
        return "Марка: " + self.brand + ", Цвет: " + self.color + ", Пробег: " +
str(self.mileage) + " км"

    def __str__(self):
        return self.info()

    def __add__(self, distance):
        if isinstance(distance, (int, float)):
            self.mileage += distance             return self         return NotIm-
plemented

    def drive(self, distance):
        self.mileage += distance         return "Машина проехала " + str(distance)
+ " км. Общий пробег: " + str(self.mileage) + " км"

    def is_old(self):
        if self.mileage > 100000:
            return "У машины большой пробег."         else:
```

```
            return "У машины небольшой пробег."

    @staticmethod     def convert_km_to_miles(km):
        return km * 0.621371

    @staticmethod     def get_vehicle_type():
        return "Автомобиль"

    @abstractmethod     def get_engine_type(self):
        pass

    @abstractmethod     def calculate_tax(self):
        pass


class PassengerCar(Car):
    def __init__(self, brand, color, mileage=0, engine_type="Бензиновый"):
        super().__init__(brand, color, mileage)
        self.engine_type = engine_type

    def get_engine_type(self):
        return f"Тип двигателя: {self.engine_type}"

    def calculate_tax(self):
        base_tax = 5000         if self.mileage > 150000:
            return base_tax * 0.8         return base_tax

    def info(self):
        return super().info() + f", Тип двигателя: {self.engine_type}"


class Truck(Car):
    def __init__(self, brand, color, mileage=0, engine_type="Дизельный", load_ca-
pacity=10):
        super().__init__(brand, color, mileage)
        self.engine_type = engine_type         self.load_capacity = load_capacity

    def get_engine_type(self):
        return f"Тип двигателя: {self.engine_type}"

    def calculate_tax(self):
        return 15000 + self.load_capacity * 100

    def info(self):
        return super().info() + f", Грузоподъемность: {self.load_capacity}т"

    def load_cargo(self):
        return f"{self.brand}: Погружено {self.load_capacity} тонн груза"


class Motorcycle(Car):
    def __init__(self, brand, color, mileage=0, engine_type="Бензиновый"):
        super().__init__(brand, color, mileage)
        self.engine_type = engine_type

    def get_engine_type(self):
        return f"Тип двигателя: {self.engine_type}"
```

```
    def calculate_tax(self):
        return 2000

    def info(self):
        return super().info() + f", Тип: Мотоцикл"

    def do_wheelie(self):
        return f"{self.brand}: Мотоцикл делает вилли!"


class Bus(Car):
    def __init__(self, brand, color, mileage=0, engine_type="Дизельный", passen-
ger_capacity=50):
        super().__init__(brand, color, mileage)
        self.engine_type = engine_type         self.passenger_capacity = passen-
ger_capacity

    def get_engine_type(self):
        return f"Тип двигателя: {self.engine_type}"

    def calculate_tax(self):
        return 12000 + self.passenger_capacity * 50

    def info(self):
        return super().info() + f", Вместимость: {self.passenger_capacity} пассажи-
ров"

    def announce_stop(self):
        return f"{self.brand}: Следующая остановка!"


print("Добро пожаловать в систему учета транспортных средств!")

while True:
    print("\n" + "=" * 50)
    print("СОЗДАНИЕ ТРАНСПОРТНОГО СРЕДСТВА")
    print("=" * 50)
    print("Выберите тип транспортного средства:")
    print("1. Легковой автомобиль")
    print("2. Грузовик")
    print("3. Мотоцикл")
    print("4. Автобус")
    print("0. Выйти")

    type_choice = input("Ваш выбор: ")

    if type_choice == "0":
        print("Выход из программы.")
        break

    print("\nВведите данные о транспорте:")
    brand = input("Марка: ")
    color = input("Цвет: ")
    mileage = int(input("Пробег (в км): "))

    if type_choice == "1":
        engine_type = input("Тип двигателя: ")
        vehicle = PassengerCar(brand, color, mileage, engine_type)
```

```
    elif type_choice == "2":
        engine_type = input("Тип двигателя: ")
        load_capacity = float(input("Грузоподъемность (тонн): "))
        vehicle = Truck(brand, color, mileage, engine_type, load_capacity)
    elif type_choice == "3":
        engine_type = input("Тип двигателя: ")
        vehicle = Motorcycle(brand, color, mileage, engine_type)
    elif type_choice == "4":
        engine_type = input("Тип двигателя: ")
        passenger_capacity = int(input("Вместимость пассажиров: "))
        vehicle = Bus(brand, color, mileage, engine_type, passenger_capacity)
    else:
        print("Неверный выбор")
        continue

    print(f"\nТранспортное средство создано: {vehicle.info()}")


    while True:
        print("\n" + "=" * 50)
        print(f"МЕНЮ ДЕЙСТВИЙ: {vehicle.brand}")
        print("=" * 50)
        print("1. Проехать расстояние")
        print("2. Проверить пробег")
        print("3. Показать информацию")
        print("4. Конвертировать пробег в мили")
        print("5. Рассчитать налог")
        print("6. Показать тип двигателя")

        if isinstance(vehicle, Truck):
            print("7. Погрузить груз")
        elif isinstance(vehicle, Motorcycle):
            print("7. Сделать вилли")
        elif isinstance(vehicle, Bus):
            print("7. Объявить остановку")
        elif isinstance(vehicle, PassengerCar):
            print("7. Информация о машине")

        print("8. Создать новое транспортное средство")
        print("0. Выйти из программы")

        action = input("Ваш выбор: ")

        if action == "1":
            distance = int(input("Сколько км проехать? "))
            print(vehicle.drive(distance))
        elif action == "2":
            print(vehicle.is_old())
        elif action == "3":
            print(vehicle.info())
        elif action == "4":
            miles = Car.convert_km_to_miles(vehicle.mileage)
            print(f"Пробег {vehicle.mileage} км = {miles:.2f} миль")
        elif action == "5":
            tax = vehicle.calculate_tax()
            print(f"Налог на транспорт: {tax} руб.")
        elif action == "6":
            print(vehicle.get_engine_type())
```

```
        elif action == "7":
            if isinstance(vehicle, Truck):
                print(vehicle.load_cargo())
            elif isinstance(vehicle, Motorcycle):
                print(vehicle.do_wheelie())
            elif isinstance(vehicle, Bus):
                print(vehicle.announce_stop())
            elif isinstance(vehicle, PassengerCar):
                print(vehicle.info())
        elif action == "8":
            break         elif action == "0":
            print("Выход из программы.")
            exit()
        else:
            print("Неверный выбор")
```

```
СОЗДАНИЕ ТРАНСПОРТНОГО СРЕДСТВА
Выберите тип транспортного средства:
1. Легковой автомобиль
2. Грузовик
3. Мотоцикл
4. Автобус
0. Выйти
Ваш выбор: 2

Введите данные о транспорте:
Марка: BMW
Цвет: белый
Пробег (в км): 20000
Тип двигателя: Дизельный
Грузоподъемность (тонн): 20
```

Рисунок 1 – Пример реализации одного из дочерних классов

5. Контрольные вопросы

1. Что дает наследование классов?

Наследование позволяет создавать новые классы на основе существующих, повторно использовать код, расширять функциональность и упрощать поддержку программ.

2. Как организовать наследование классов в языке Питон?

В Python наследование задаётся указанием родительского класса в скобках при объявлении:

```
1. class Parent:
2.     pass
3. class Child(Parent):
4.     pass
```